

Lecture 14: IP Addressing, CIDR, and Subnetting

How addresses are structured, allocated, and divided

EC 441 — Introduction to Computer Networking
Boston University

Spring 2026

Learning Objectives

By the end of this lecture, you should be able to:

- Explain why an IP address is split into a **network portion** and a **host portion**, and what that split enables
- Describe **classful addressing** (Class A/B/C) and explain why it was abandoned
- Interpret **CIDR prefix notation** (e.g., 192.168.1.0/24) and relate it to a subnet mask
- Compute the network address, broadcast address, and usable host range for a given prefix
- Divide an address block into subnets using a longer prefix length
- Identify the major **special-purpose** IPv4 ranges (RFC 1918, loopback, link-local)
- Describe the **IANA** → **RIR** → **ISP** → **org** allocation hierarchy and explain why IPv4 exhaustion occurred
- Use Python's `ipaddress` module to perform address and subnet arithmetic

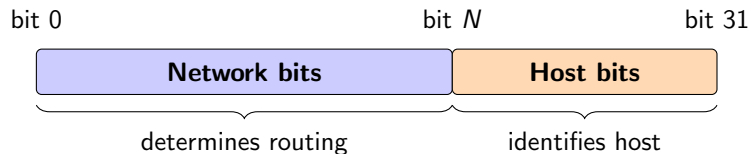
Key Acronyms

Acronym	Meaning
APIPA	Automatic Private IP Addressing
ARIN	Am. Registry for Internet Numbers
CGN	Carrier-Grade NAT
CIDR	Classless Inter-Domain Routing
DHCP	Dynamic Host Configuration Protocol
IANA	Internet Assigned Numbers Authority

Acronym	Meaning
NAT	Network Address Translation
RFC	Request for Comments
RIR	Regional Internet Registry
RIPE	Réseaux IP Européens NCC
VLSM	Variable-Length Subnet Masking
VPC	Virtual Private Cloud

The Core Mental Model: One Address, Two Jobs

An IPv4 address is 32 bits that encode **two pieces of information**:



The **prefix length** /N says how many bits are network bits.

Why this matters — hierarchical routing:

Internet core: 128.197.0.0/16 → “toward BU”

BU edge router: 128.197.10.0/24 → “toward Photonics”

Building switch: 128.197.10.42 → port 7

Each level knows only what it needs to. A router in Tokyo does not need to know about individual hosts at BU — just which direction to send BU-destined packets.

IPv4: 32 Bits, Dotted-Decimal Notation

IPv4 uses a **32-bit address space**: $2^{32} \approx 4.3$ billion addresses total.

Dotted-decimal: the 32 bits are split into four 8-bit octets, each written 0–255:

Binary: 10000000.11000101.00001010.00101010

Dotted-decimal: 128 .197 .10 .42

(This is 128.197.10.42 — a host on BU's network.)

Key conversions

Decimal	Binary	Decimal	Binary
0	0000 0000	192	1100 0000
128	1000 0000	224	1110 0000
255	1111 1111	240	1111 0000

These come up constantly in masks.

Dotted-decimal is purely a human convenience — router hardware works on bits.

[Wikipedia: IPv4]

Classful Addressing (1981–1993)

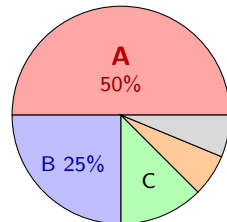
Class	Bits	Prefix	Hosts/net	Range
A	0	/8	16.7M	0.0.0.0 – 127.x.x.x
B	10	/16	65,534	128.x.x.x – 191.x.x.x
C	110	/24	254	192.x.x.x – 223.x.x.x
D	1110	—	multicast	224.x.x.x – 239.x.x.x
E	1111	—	reserved	240.x.x.x – 255.x.x.x

Reading an address: first octet 128–191 $\Rightarrow$ Class B, prefix /16, first two octets are the network.

128.197.10.42 $\rightarrow$ net 128.197.0.0, host .10.42

BU's 128.197.0.0/16 is a Class B block from the classful era.

IPv4 space by class:



D (multicast) + E (reserved) = 12.5%

Class A = **half** of all
IPv4 address space

[Wikipedia: Classful network]

The Class A Lottery: Who Got a /8?

In the 1980s–90s, organizations could apply for a Class A /8 block — **16.7 million addresses**. At today's transfer prices (\$40/addr), a /8 $\approx$ **\$650 million**.

Organization	Block	Status
US Dept. of Defense	6, 7, 11, 21, 22, 26, ...	Retained (~13 /8s)
IBM	9.0.0.0/8	Retained
AT&T	12.0.0.0/8	Retained
Apple	17.0.0.0/8	Retained
MIT	18.0.0.0/8	Partial — sold half to Amazon (2017, ~\$4M)
Ford Motor Company	19.0.0.0/8	Retained
General Electric	3.0.0.0/8	Transferred → Amazon AWS

The punchline

Ford makes cars. MIT runs a university. Neither particularly needs 16 million public IP addresses. These allocations reflect the era: addresses seemed unlimited, and the scale of the future internet was unimaginable.

[Wikipedia: List of assigned /8 blocks]

Why Classful Addressing Failed (1): Address Waste

Problem 1: Waste at every scale

The binary choice — Class B (65K hosts) or Class C (254 hosts) — matched almost no real organization's needs:

Organization	Hosts needed	Class assigned	Addresses wasted
Small company (50 hosts)	50	C (/24, 254)	~200 manageable
Medium company (2K hosts)	2000	B (/16, 65K)	63,000
BU (5K students)	5000	B (/16, 65K)	60,000
Large ISP (1M hosts)	1M+	A (/8, 16.7M)	15,700,000

By 1993, roughly **half** of all assigned IPv4 address space sat unused inside organizations that had been given overly large blocks.

Why Classful Addressing Failed (2): Routing Explosion

Problem 2: Routing table explosion

A medium company needing 2000 hosts but denied a Class B received **8 separate Class C blocks**. Each required its own entry in the global routing table — no aggregation was possible under classful rules.

8 separate entries: /24 /24 /24 /24 /24 /24 /24 /24

By the early 1990s the global routing table was growing **exponentially**. Router hardware could not keep up.

The core mismatch

Class B = 65K hosts. Class C = 254 hosts. Medium organizations fell into neither category — they needed multiple Class C blocks that could not be aggregated.

CIDR: The Fix (RFC 1519, 1993)

Classless Inter-Domain Routing made three changes:

- ① **Any prefix length is valid** — /19, /22, /27, whatever fits the network
- ② **Prefix length is written explicitly** — 192.168.1.0/24, not “Class C, therefore /24”
- ③ **Aggregation is first-class** — adjacent blocks with a common prefix collapse into one shorter-prefix routing entry (*supernetting*)

Before CIDR — 8 separate routing entries:

192.168.H.0/24
192.168.G.0/24
192.168.F.0/24
192.168.E.0/24
192.168.D.0/24
192.168.C.0/24
192.168.B.0/24
192.168.A.0/24

After CIDR — 1 aggregated entry:

192.168.0.0/21
covers all 8 /24s (2048 addresses)

CIDR reduced the global routing table from an explosive trajectory to a manageable one — bought the internet roughly a decade before IPv4 exhaustion became critical.

[Wikipedia: CIDR]

Subnet Masks and Prefix Lengths

A CIDR prefix is equivalent to a **subnet mask**: N ones followed by $(32 - N)$ zeros.

```
/24 -> 11111111.11111111.11111111.00000000 -> 255.255.255.0
/20 -> 11111111.11111111.11110000.00000000 -> 255.255.240.0
/26 -> 11111111.11111111.11111111.11000000 -> 255.255.255.192
/30 -> 11111111.11111111.11111111.11111100 -> 255.255.255.252
```

Key calculations for a $/N$ prefix:

Prefix	Total addresses	Usable hosts	Typical use
$/30$	4	2	Point-to-point links
$/26$	64	62	Small office
$/24$	256	254	Standard LAN segment
$/22$	1,024	1,022	Large campus segment
$/16$	65,536	65,534	Large organization

Usable hosts = $2^{32-N} - 2$ (subtract network address and broadcast address)

[Wikipedia: Subnet mask]

Network Address, Broadcast, and Host Range

Given an address and prefix, three things to compute:

Network address = address AND mask (all host bits zeroed):

```
192.168.1.75 = 11000000.10101000.00000001.01001011
& 255.255.255.0 = 11111111.11111111.11111111.00000000
-----
11000000.10101000.00000001.00000000 = 192.168.1.0
```

Broadcast address = all host bits set to 1:

192.168.1.0/24 → broadcast = 192.168.1.255

Usable host range: network + 1 through broadcast - 1:

192.168.1.0/24 → hosts 192.168.1.1 – 192.168.1.254 (254 hosts)

The alignment rule

A valid subnet's network address must be a **multiple of the block size**. For a /26 (block size 64): valid network addresses are .0, .64, .128, .192. 192.168.1.10/26 is **invalid** — .10 is not a multiple of 64.

Demo: ip addr show

Demo: ip addr show (Linux) / ifconfig (macOS)

Read interface addresses and prefix lengths directly from the OS.

```
$ ip addr show
1: lo: <LOOPBACK,UP>
    inet 127.0.0.1/8 scope host lo

2: eth0: <BROADCAST,MULTICAST,UP>
    inet 192.168.1.42/24 brd 192.168.1.255 scope global eth0

3: eth1: <BROADCAST,MULTICAST,UP>
    inet 10.0.5.1/16 brd 10.0.255.255 scope global eth1
```

- 127.0.0.1/8 — loopback; entire 127.x.x.x routes locally
- 192.168.1.42/24 — host .42 on the 192.168.1.0/24 network; broadcast .255
- 10.0.5.1/16 — host on a /16; broadcast is 10.0.255.255 (all host bits set)
- brd field confirms the broadcast address the kernel computed

Subnetting: Dividing an Address Block

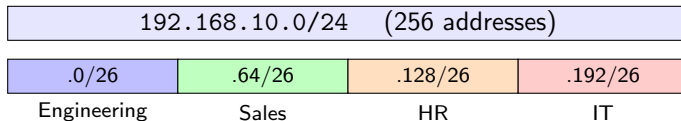
The problem: your organization has 192.168.10.0/24. You want separate network segments for Engineering, Sales, HR, and IT — different broadcast domains, different firewall rules.

The solution: subnet — extend the prefix to create smaller blocks.

Rule: borrow k bits $\Rightarrow 2^k$ subnets, each a $/ (N+k)$

Example: divide 192.168.10.0/24 **into 4 subnets**

Borrow 2 bits ($2^2 = 4$). New prefix: $24 + 2 = 26$. Each /26 has $2^6 = 64$ addresses, 62 usable.



Check: $4 \times 64 = 256$ — the entire /24 is accounted for.

[Wikipedia: Subnetwork]

Subnetting: The Bit-Level View

192.168.10.0/24 → **four /26s** — borrow 2 bits, new prefix /26:

```
Original /24: 192.168.10. 00|000000 (network|host, | at bit 26)
               ^^ borrowed bits determine subnet number
```

```
Subnet 0: 192.168.10.00|000000 -> 192.168.10.0/26 (.0 - .63)
```

```
Subnet 1: 192.168.10.01|000000 -> 192.168.10.64/26 (.64 - .127)
```

```
Subnet 2: 192.168.10.10|000000 -> 192.168.10.128/26 (.128 - .191)
```

```
Subnet 3: 192.168.10.11|000000 -> 192.168.10.192/26 (.192 - .255)
```

Alignment rule

Each subnet's network address must be a **multiple of the block size**. For /26 (block size 64): valid starts are .0, .64, .128, .192 only. 192.168.10.10/26 is invalid — .10 is not a multiple of 64.

Subnet Membership Check

Are two hosts in the same subnet? AND each address with the mask and compare:

192.168.10.75 and **192.168.10.100** — same /26?

```
.75 AND 255.255.255.192 -> 192.168.10.64  
.100 AND 255.255.255.192 -> 192.168.10.64 <- same network -> same subnet
```

192.168.10.75 and **192.168.10.130** — same /26?

```
.75 AND 255.255.255.192 -> 192.168.10.64  
.130 AND 255.255.255.192 -> 192.168.10.128 <- different -> must route
```

Key implication

Hosts in different subnets cannot communicate directly — traffic between them must pass through a **router**, even if both hosts are on the same floor of the same building. Subnet boundaries are logical, not physical.

Variable-Length Subnet Masking (VLSM)

Subnets within an organization don't need to be equal-sized. **VLSM** allows different prefix lengths in different parts of the address space.

Example: BU ECE department, assigned 192.168.20.0/24

Use	Hosts needed	Prefix	Block
Lab network	120	/25 (126 usable)	192.168.20.0/25
Faculty offices	50	/26 (62 usable)	192.168.20.128/26
Staff	20	/27 (30 usable)	192.168.20.192/27
Router link	2	/30 (2 usable)	192.168.20.224/30

Avoid overlaps

VLSM is efficient but requires careful bookkeeping. Always verify that no two assigned blocks overlap. The alignment rule keeps things tidy: each block must start at a multiple of its block size.

[Wikipedia: VLSM]

RFC 1918: Private Address Space

RFC 1918 (1996) designated three blocks as **private address space** — never assigned to hosts on the public internet:

Block	Range	Size	Typical use
10.0.0.0/8	10.0.0.0 – 10.255.255.255	16M	Enterprise, cloud VPCs
172.16.0.0/12	172.16.0.0 – 172.31.255.255	1M	Medium enterprise
192.168.0.0/16	192.168.0.0 – 192.168.255.255	65K	Home networks, small offices

Private addresses are:

- **Not routable** on the public internet — border routers drop packets with private source/destination
- **Reusable** — thousands of organizations can all use 192.168.1.0/24 without conflict; they are never visible outside their own network
- **Used behind NAT** — a single public IP can represent thousands of private hosts (covered in L18)

172.17.0.0/16 is Docker's default container network (a subdivision of the /12).

[Wikipedia: Private network]

Other Special-Purpose Ranges

Range	Purpose	Notes
127.0.0.0/8	Loopback	127.0.0.1 = localhost; packets never leave host
169.254.0.0/16	Link-local (APIPA)	Auto-assigned when DHCP fails; not routable beyond link
224.0.0.0/4	Multicast	OSPF: 224.0.0.5/6; RIP: 224.0.0.9
255.255.255.255	Limited broadcast	All hosts on local subnet; not forwarded by routers
0.0.0.0	"This host"	Used in DHCP Discover before address is assigned
100.64.0.0/10	Carrier-Grade NAT	RFC 6598; ISP-internal; not on public internet

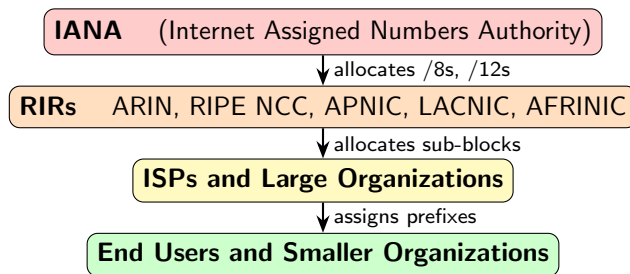
You will see these in practice:

- 192.168.x.x — your laptop's address right now (home router DHCP)
- 10.x.x.x — AWS/GCP/Azure VPCs; your organization's corporate network
- 169.254.x.x — DHCP failed; check your network config
- 100.64.x.x — may appear in traceroute through certain ISPs

[Wikipedia: Link-local address] [Wikipedia: CGN]

The Allocation Hierarchy

IPv4 addresses are allocated through a **four-level hierarchy**:



ARIN	North America
RIPE NCC	Europe, Middle East, Central Asia
APNIC	Asia-Pacific
LACNIC	Latin America and Caribbean
AFRINIC	Africa

[Wikipedia: RIR]

IPv4 Exhaustion

$2^{32} \approx 4.3$ billion addresses. The internet grew faster than anyone predicted.

Event	Date
IANA exhausts its free pool	February 2011
APNIC (Asia-Pacific) exhausts	April 2011
RIPE NCC (Europe) exhausts	September 2012
LACNIC (Latin America) exhausts	June 2014
ARIN (North America) exhausts	September 2015
AFRINIC (Africa) exhausts	2021

New IPv4 addresses are no longer available from RIRs. They are bought on the **transfer market** at **\$30–50 per address** (\$8K–12K for a /24).

What kept IPv4 alive: NAT (covered in L18)

One public IP representing thousands of private hosts extended IPv4 life by ~ 20 years beyond original projections.

[Wikipedia: IPv4 exhaustion]

Demo: Python ipaddress stdlib — no install needed

```
import ipaddress

net = ipaddress.IPv4Network("192.168.10.0/24")

print(net.network_address) # 192.168.10.0
print(net.broadcast_address) # 192.168.10.255
print(net.netmask) # 255.255.255.0
print(net.num_addresses) # 256

# Usable hosts (excludes network and broadcast)
hosts = list(net.hosts())
print(hosts[0]) # 192.168.10.1
print(hosts[-1]) # 192.168.10.254
print(len(hosts)) # 254
```

[Wikipedia: Subnetting]

Python: Subnetting with .subnets()

```
import ipaddress

# Divide 192.168.10.0/24 into four /26s
net = ipaddress.IPv4Network("192.168.10.0/24")
subnets = list(net.subnets(prefixlen_diff=2)) # add 2 bits -> /26

for i, s in enumerate(subnets):
    hosts = list(s.hosts())
    print(f"Subnet_{i}:_{s}_{hosts}_{hosts[0]}--_{hosts[-1]}")
```

```
Subnet 0: 192.168.10.0/26 hosts 192.168.10.1 -- 192.168.10.62
Subnet 1: 192.168.10.64/26 hosts 192.168.10.65 -- 192.168.10.126
Subnet 2: 192.168.10.128/26 hosts 192.168.10.129 -- 192.168.10.190
Subnet 3: 192.168.10.192/26 hosts 192.168.10.193 -- 192.168.10.254
```

```
# Containment check: which subnet is 192.168.10.75 in?
host = ipaddress.IPv4Address("192.168.10.75")
for s in subnets:
    if host in s:
        print(f"{host}_is_in_{s}") # -> 192.168.10.75 is in 192.168.10.64/26
```

Python: Special Address Checks

```
import ipaddress

addrs = [
    "10.0.0.1", # RFC 1918
    "192.168.1.42", # RFC 1918
    "127.0.0.1", # loopback
    "169.254.1.1", # link-local (APIPA)
    "224.0.0.5", # multicast (OSPF)
]

for s in addrs:
    a = ipaddress.IPv4Address(s)
    print(f"{s:16s}__private={a.is_private}"
          f"__loopback={a.is_loopback}"
          f"__link_local={a.is_link_local}"
          f"__multicast={a.is_multicast}")
```

Note: `is_private` returns True for RFC 1918 *and* other IANA special-use ranges (loopback, link-local, etc.) — use not `a.is_global` for a broader “not public” test.

Wireshark: RFC 1918 vs. Public Addresses

Wireshark: spot private vs. public addresses in browser traffic

- 1 Start a capture on your WiFi interface
- 2 Open a webpage (e.g., google.com)
- 3 Filter: `ip` to see all IPv4 traffic

What you will see:

- **Source:** your laptop's RFC 1918 address (192.168.x.x)
- **Destination:** the server's public IP (142.250.x.x for Google)
- **Return traffic:** source = public IP, destination = your private IP

The NAT question

Your 192.168.x.x source is not routable on the internet — how does Google know where to reply? NAT at your router rewrites the source to a public IP. Covered in L18.

Filter: `ip.src == 192.168.0.0/16 || ip.src == 10.0.0.0/8`

What's Next

Today completed the addressing half of the network layer story: IP addresses are hierarchically structured, allocated from IANA through RIRs, and subdivided via CIDR and subnetting.

The forwarding table entries we have been working with now have a full interpretation. 10.0.0.0/8, 192.168.1.0/24, 0.0.0.0/0 — you know where these come from and how to do the arithmetic.

The remaining question: how does the forwarding table get built?

- ① **L15 (Tue Mar 24):** Link-State routing and Dijkstra's algorithm — every router builds a complete map of the network and computes shortest paths
- ② **L16 (Thu Mar 26):** Distance-Vector routing and Bellman-Ford — routers share only their distance estimates, not the full map
- ③ **L17 (Tue Mar 31):** Autonomous Systems and BGP — how routing works between organizations
- ④ **L18 (Thu Apr 02):** IPv4 datagram format, NAT, IPv6, DHCP